

4.1 Summary of clocking and power control functions

This section describes the generation of the various clocks needed by the LPC176x/5x and options of clock source selection, as well as power control and wake-up from reduced power modes. Functions described in the following subsections include:

- Oscillators
- Clock source selection
- PLLs
- Clock dividers
- APB dividers
- Power control
- Wake-up timer
- External clock output

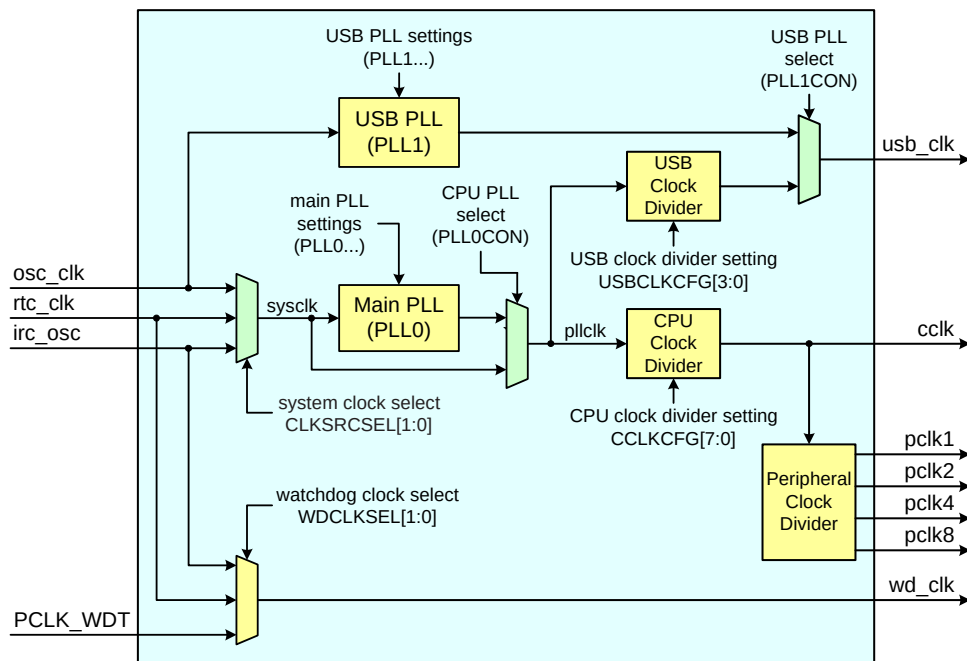


Fig 7. Clock generation for the LPC176x/5x

4.2 Register description

All registers, regardless of size, are on word address boundaries. Details of the registers appear in the description of each function.

Table 14. Summary of system control registers

Name	Description	Access	Reset value	Address
Clock source selection				
CLKSRCSEL	Clock Source Select Register	R/W	0	0x400F C10C
Phase Locked Loop (PLL0, Main PLL)				
PLL0CON	PLL0 Control Register	R/W	0	0x400F C080
PLL0CFG	PLL0 Configuration Register	R/W	0	0x400F C084
PLL0STAT	PLL0 Status Register	RO	0	0x400F C088
PLL0FEED	PLL0 Feed Register	WO	NA	0x400F C08C
Phase Locked Loop (PLL1, USB PLL)				
PLL1CON	PLL1 Control Register	R/W	0	0x400F C0A0
PLL1CFG	PLL1 Configuration Register	R/W	0	0x400F C0A4
PLL1STAT	PLL1 Status Register	RO	0	0x400F C0A8
PLL1FEED	PLL1 Feed Register	WO	NA	0x400F C0AC
Clock dividers				
CCLKCFG	CPU Clock Configuration Register	R/W	0	0x400F C104
USBCLKCFG	USB Clock Configuration Register	R/W	0	0x400F C108
PCLKSEL0	Peripheral Clock Selection register 0.	R/W	0	0x400F C1A8
PCLKSEL1	Peripheral Clock Selection register 1.	R/W	0	0x400F C1AC
Power control				
PCON	Power Control Register	R/W	0	0x400F C0C0
PCONP	Power Control for Peripherals Register	R/W	0x03BE	0x400F C0C4
Utility				
CLKOUTCFG	Clock Output Configuration Register	R/W	0	0x400F C1C8

4.3 Oscillators

The LPC176x/5x includes three independent oscillators. These are the Main Oscillator, the Internal RC Oscillator, and the RTC oscillator. Each oscillator can be used for more than one purpose as required in a particular application. This can be seen in [Figure 7](#).

Following Reset, the LPC176x/5x will operate from the Internal RC Oscillator until switched by software. This allows systems to operate without any external crystal, and allows the boot loader code to operate at a known frequency.

4.3.1 Internal RC oscillator

The Internal RC Oscillator (IRC) may be used as the clock source for the watchdog timer, and/or as the clock that drives PLL0 and subsequently the CPU. The precision of the IRC does not allow for use of the USB interface, which requires a much more precise time base in order to comply with the USB specification. Also, the IRC should not be used with the CAN1/2 block if the CAN baud rate is higher than 100 kbit/s. The nominal IRC frequency is 4 MHz.

Upon power-up or any chip reset, the LPC176x/5x uses the IRC as the clock source. Software may later switch to one of the other available clock sources.

4.3.2 Main oscillator

The main oscillator can be used as the clock source for the CPU, with or without using PLL0. The main oscillator operates at frequencies of 1 MHz to 25 MHz. This frequency can be boosted to a higher frequency, up to the maximum CPU operating frequency, by the Main PLL (PLL0). The oscillator output is called OSC_CLK. The clock selected as the PLL0 input is PLLCLKIN and the ARM processor clock frequency is referred to as CCLK for purposes of rate equations, etc. elsewhere in this document. The frequencies of PLLCLKIN and CCLK are the same value unless the PLL0 is active and connected. Refer to [Section 4.5 “PLL0 \(Phase Locked Loop 0\)”](#) for details.

The on-board oscillator in the LPC176x/5x can operate in one of two modes: slave mode and oscillation mode.

In slave mode the input clock signal should be coupled by means of a capacitor of 100 pF (C_C in [Figure 8](#), drawing a), with an amplitude between 200 mVrms and 1000 mVrms. This corresponds to a square wave signal with a signal swing of between 280 mV and 1.4 V. The XTAL2 pin in this configuration can be left unconnected.

External components and models used in oscillation mode are shown in [Figure 8](#), drawings b and c, and in [Table 15](#) and [Table 16](#). Since the feedback resistance is integrated on chip, only a crystal and the capacitances C_{X1} and C_{X2} need to be connected externally in case of fundamental mode oscillation (the fundamental frequency is represented by L , C_L and R_S). Capacitance C_P in [Figure 8](#), drawing c, represents the parallel package capacitance and should not be larger than 7 pF. Parameters F_C , C_L , R_S and C_P are supplied by the crystal manufacturer.

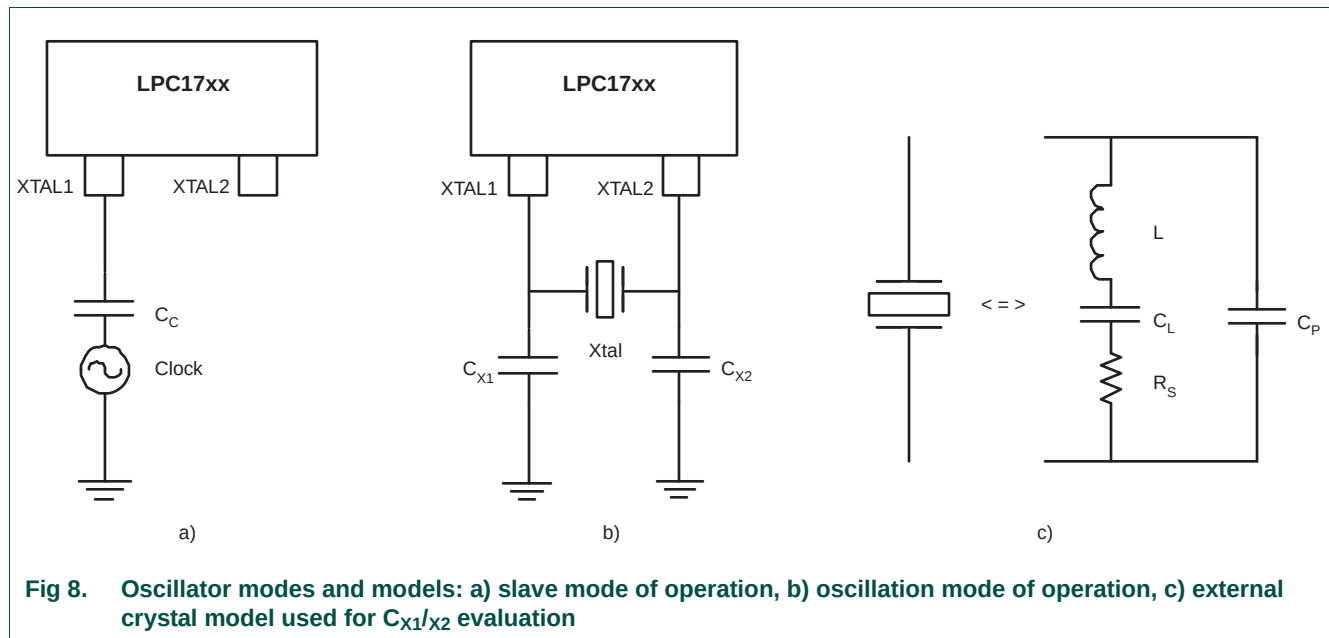


Table 15. Recommended values for $C_{X1/X2}$ in oscillation mode (crystal and external components parameters) low frequency mode (OSCRANGE = 0, see [Table 13](#))

Fundamental oscillation frequency F_{Osc}	Crystal load capacitance C_L	Maximum crystal series resistance R_S	External load capacitors C_{X1}, C_{X2}
1 MHz to 5 MHz	10 pF	< 300 Ω	18 pF, 18 pF
	20 pF	< 300 Ω	39 pF, 39 pF
	30 pF	< 300 Ω	57 pF, 57 pF
5 MHz to 10 MHz	10 pF	< 300 Ω	18 pF, 18 pF
	20 pF	< 200 Ω	39 pF, 39 pF
	30 pF	< 100 Ω	57 pF, 57 pF
10 MHz to 15 MHz	10 pF	< 160 Ω	18 pF, 18 pF
	20 pF	< 60 Ω	39 pF, 39 pF
15 MHz to 20 MHz	10 pF	< 80 Ω	18 pF, 18 pF

Table 16. Recommended values for $C_{X1/X2}$ in oscillation mode (crystal and external components parameters) high frequency mode (OSCRANGE = 1, see [Table 13](#))

Fundamental oscillation frequency F_{Osc}	Crystal load capacitance C_L	Maximum crystal series resistance R_S	External load capacitors C_{X1}, C_{X2}
15 MHz to 20 MHz	10 pF	< 180 Ω	18 pF, 18 pF
	20 pF	< 100 Ω	39 pF, 39 pF
20 MHz to 25 MHz	10 pF	< 160 Ω	18 pF, 18 pF
	20 pF	< 80 Ω	39 pF, 39 pF

Since chip operation always begins using the Internal RC Oscillator, and the main oscillator may not be used at all in some applications, it will only be started by software request. This is accomplished by setting the OSCEN bit in the SCS register, as described in [Table 13](#). The main oscillator provides a status flag (the OSCSTAT bit in the SCS register) so that software can determine when the oscillator is running and stable. At that

point, software can control switching to the main oscillator as a clock source. Prior to starting the main oscillator, a frequency range must be selected by configuring the OSCRANGE bit in the SCS register.

4.3.3 RTC oscillator

The RTC oscillator provides a 1 Hz clock to the RTC and a 32 kHz clock output that can be used as the clock source for PLL0 and CPU and/or the watchdog timer.

Remark: The RTC oscillator must not be used as a clock source when the PLL0 output is selected to drive the USB controller. In this case select the main oscillator as clock source for PLL0 (see also [Table 17](#)).

4.4 Clock source selection multiplexer

Several clock sources may be chosen to drive PLL0 and ultimately the CPU and on-chip peripheral devices. The clock sources available are the main oscillator, the RTC oscillator, and the Internal RC oscillator.

The clock source selection can only be changed safely when PLL0 is not connected. For a detailed description of how to change the clock source in a system using PLL0 see [Section 4.5.13 “PLL0 setup sequence”](#).

Note the following restrictions regarding the choice of clock sources:

- Only the main oscillator must be used (via PLL0) as the clock source for the USB subsystem. The IRC or RTC oscillators do not provide the proper tolerances for this use.
- The IRC oscillator should not be used (via PLL0) as the clock source for the CAN controllers if the CAN baud rate is higher than 100 kbit/s.

4.4.1 Clock Source Select register (CLKSRCSEL - 0x400F C10C)

The CLKSRCSEL register contains the bits that select the clock source for PLL0.

Table 17. Clock Source Select register (CLKSRCSEL - address 0x400F C10C) bit description

Bit	Symbol	Value	Description	Reset value
1:0	CLKSRC		Selects the clock source for PLL0 as follows:	0
		00	Selects the Internal RC oscillator as the PLL0 clock source (default).	
		01	Selects the main oscillator as the PLL0 clock source. Remark: Select the main oscillator as PLL0 clock source if the PLL0 clock output is used for USB or for CAN with baudrates > 100 kBit/s.	
		10	Selects the RTC oscillator as the PLL0 clock source.	
		11	Reserved, do not use this setting.	
Warning: Improper setting of this value, or an incorrect sequence of changing this value may result in incorrect operation of the device.				
31:2	-	0	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

4.5 PLL0 (Phase Locked Loop 0)

PLL0 accepts an input clock frequency in the range of 32 kHz to 50 MHz. The clock source is selected in the CLKSRCSEL register (see [Section 4.4](#)). The input frequency is multiplied up to a high frequency, then divided down to provide the actual clock used by the CPU, peripherals, and optionally the USB subsystem. Note that the USB subsystem has its own dedicated PLL (see [Section 4.6](#)). PLL0 can produce a clock up to the maximum allowed for the CPU, which is 120 MHz on high speed versions (LPC1769 and LPC1759), and 100 MHz on other versions.

4.5.1 PLL0 operation

The PLL input, in the range of 32 kHz to 50 MHz, may initially be divided down by a value "N", which may be in the range of 1 to 256. This input division provides a greater number of possibilities in providing a wide range of output frequencies from the same input frequency.

Following the PLL input divider is the PLL multiplier. This can multiply the input divider output through the use of a Current Controlled Oscillator (CCO) by a value "M", in the range of 6 through 512, plus additional values listed in [Table 21](#). The resulting frequency must be in the range of 275 MHz to 550 MHz. The multiplier works by dividing the CCO output by the value of M, then using a phase-frequency detector to compare the divided CCO output to the multiplier input. The error value is used to adjust the CCO frequency.

There are additional dividers at the output of PLL0 to bring the frequency down to what is needed for the CPU, peripherals, and potentially the USB subsystem. PLL0 output dividers are described in the Clock Dividers section following the PLL0 description. A block diagram of PLL0 is shown in [Figure 9](#)

PLL activation is controlled via the PLL0CON register. PLL0 multiplier and divider values are controlled by the PLL0CFG register. These two registers are protected in order to prevent accidental alteration of PLL0 parameters or deactivation of the PLL. Since all chip operations, including the Watchdog Timer, could be dependent on PLL0 if so configured (for example when it is providing the chip clock), accidental changes to the PLL0 setup values could result in unexpected or fatal behavior of the microcontroller. The protection is accomplished by a feed sequence similar to that of the Watchdog Timer. Details are provided in the description of the PLL0FEED register.

PLL0 is turned off and bypassed following a chip Reset and by entering Power-down mode. PLL0 must be configured, enabled, and connected to the system by software.

It is important that the setup procedure described in [Section 4.5.13 "PLL0 setup sequence"](#) is followed or PLL0 might not operate at all!

4.5.1.1 PLL0 and startup/boot code interaction

When there is no valid user code (determined by the checksum word) in the user flash or the ISP enable pin (P2.10) is pulled low on startup, the ISP mode will be entered and the boot code will setup the PLL with the IRC. Therefore it can not be assumed that the PLL is disabled when the user opens a debug session to debug the application code. The user startup code must follow the steps described in this chapter to disconnect the PLL.

4.5.2 PLL0 register description

PLL0 is controlled by the registers shown in [Table 18](#). More detailed descriptions follow.

Warning: Improper setting of PLL0 values may result in incorrect operation of the device!

Table 18. PLL0 registers

Name	Description	Access	Reset value ^[1]	Address
PLL0CON	PLL0 Control Register. Holding register for updating PLL0 control bits. Values written to this register do not take effect until a valid PLL0 feed sequence has taken place.	R/W	0	0x400F C080
PLL0CFG	PLL0 Configuration Register. Holding register for updating PLL0 configuration values. Values written to this register do not take effect until a valid PLL0 feed sequence has taken place.	R/W	0	0x400F C084
PLL0STAT	PLL0 Status Register. Read-back register for PLL0 control and configuration information. If PLL0CON or PLL0CFG have been written to, but a PLL0 feed sequence has not yet occurred, they will not reflect the current PLL0 state. Reading this register provides the actual values controlling the PLL0, as well as the PLL0 status.	RO	0	0x400F C088
PLL0FEED	PLL0 Feed Register. This register enables loading of the PLL0 control and configuration information from the PLL0CON and PLL0CFG registers into the shadow registers that actually affect PLL0 operation.	WO	NA	0x400F C08C

[1] Reset Value reflects the data stored in used bits only. It does not include reserved bits content.

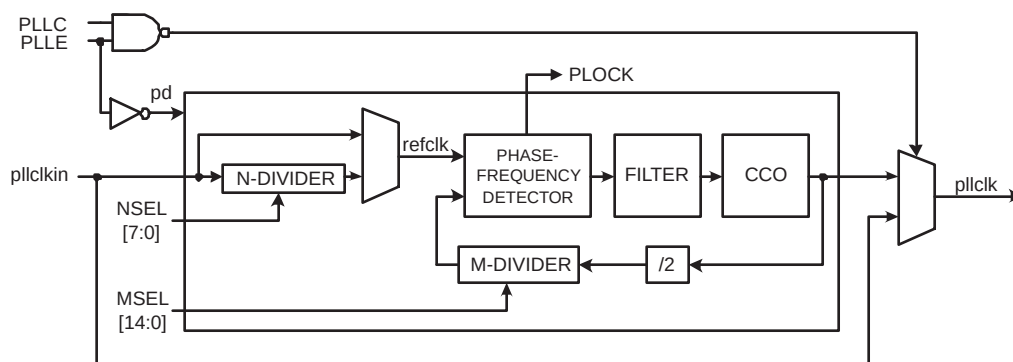


Fig 9. PLL0 block diagram

4.5.3 PLL0 Control register (PLL0CON - 0x400F C080)

The PLL0CON register contains the bits that enable and connect PLL0. Enabling PLL0 allows it to attempt to lock to the current settings of the multiplier and divider values. Connecting PLL0 causes the processor and most chip functions to run from the PLL0 output clock. Changes to the PLL0CON register do not take effect until a correct PLL0

feed sequence has been given (see [Section 4.5.8 “PLL0 Feed register \(PLL0FEED - 0x400F C08C\)”](#)).

Table 19. PLL Control register (PLL0CON - address 0x400F C080) bit description

Bit	Symbol	Description	Reset value
0	PLLE0	PLL0 Enable. When one, and after a valid PLL0 feed, this bit will activate PLL0 and allow it to lock to the requested frequency. See PLL0STAT register, Table 22 .	0
1	PLLC0	PLL0 Connect. Setting PLLC0 to one after PLL0 has been enabled and locked, then followed by a valid PLL0 feed sequence causes PLL0 to become the clock source for the CPU, AHB peripherals, and used to derive the clocks for APB peripherals. The PLL0 output may potentially be used to clock the USB subsystem if the frequency is 48 MHz. See PLL0STAT register, Table 22 .	0
31:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

PLL0 must be set up, enabled, and Lock established before it may be used as a clock source. When switching from the oscillator clock to the PLL0 output or vice versa, internal circuitry synchronizes the operation in order to ensure that glitches are not generated. Hardware does not insure that PLL0 is locked before it is connected or automatically disconnect PLL0 if lock is lost during operation. In the event of loss of lock on PLL0, it is likely that the oscillator clock has become unstable and disconnecting PLL0 will not remedy the situation.

4.5.4 PLL0 Configuration register (PLL0CFG - 0x400F C084)

The PLL0CFG register contains PLL0 multiplier and divider values. Changes to the PLL0CFG register do not take effect until a correct PLL feed sequence has been given (see [Section 4.5.8 “PLL0 Feed register \(PLL0FEED - 0x400F C08C\)”](#)). Calculations for the PLL frequency, and multiplier and divider values are found in the [Section 4.5.10 “PLL0 frequency calculation”](#).

Table 20. PLL0 Configuration register (PLL0CFG - address 0x400F C084) bit description

Bit	Symbol	Description	Reset value
14:0	MSEL0	PLL0 Multiplier value. Supplies the value "M" in PLL0 frequency calculations. The value stored here is M - 1. Supported values for M are 6 through 512 and those listed in Table 21 . Note: Not all values of M are needed, and therefore some are not supported by hardware. For details on selecting values for MSEL0 see Section 4.5.10 “PLL0 frequency calculation” .	0
15	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
23:16	NSEL0	PLL0 Pre-Divider value. Supplies the value "N" in PLL0 frequency calculations. The value stored here is N - 1. Supported values for N are 1 through 32. Note: For details on selecting the right value for NSEL0 see Section 4.5.10 “PLL0 frequency calculation” .	0
31:24	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

Table 21. Multiplier values for PLL0 with a 32 kHz input

Multiplier (M)	Pre-divide (N)	F _{CCO}	Multiplier (M)	Pre-divide (N)	F _{CCO}
4272	1	279.9698	12085	2	396.0013
4395	1	288.0307	12207	2	399.9990
4578	1	300.0238	12817	2	419.9875
4725	1	309.6576	12817	3	279.9916
4807	1	315.0316	13184	2	432.0133
5127	1	336.0031	13184	3	288.0089
5188	1	340.0008	13672	2	448.0041
5400	1	353.8944	13733	2	450.0029
5493	1	359.9892	13733	3	300.0020
5859	1	383.9754	13916	2	455.9995
6042	1	395.9685	14099	2	461.9960
6075	1	398.1312	14420	3	315.0097
6104	1	400.0317	14648	2	479.9857
6409	1	420.0202	15381	2	504.0046
6592	1	432.0133	15381	3	336.0031
6750	1	442.3680	15564	3	340.0008
6836	1	448.0041	15625	2	512.0000
6866	1	449.9702	15869	2	519.9954
6958	1	455.9995	16113	2	527.9908
7050	1	462.0288	16479	3	359.9892
7324	1	479.9857	17578	3	383.9973
7425	1	486.6048	18127	3	395.9904
7690	1	503.9718	18311	3	400.0099
7813	1	512.0328	19226	3	419.9984
7935	1	520.0282	19775	3	431.9915
8057	1	528.0236	20508	3	448.0041
8100	1	530.8416	20599	3	449.9920
8545	2	280.0026	20874	3	455.9995
8789	2	287.9980	21149	3	462.0070
9155	2	299.9910	21973	3	480.0075
9613	2	314.9988	23071	3	503.9937
10254	2	336.0031	23438	3	512.0109
10376	2	340.0008	23804	3	520.0063
10986	2	359.9892	24170	3	528.0017
11719	2	384.0082			

4.5.5 PLL0 Status register (PLL0STAT - 0x400F C088)

The read-only PLL0STAT register provides the actual PLL0 parameters that are in effect at the time it is read, as well as PLL0 status. PLL0STAT may disagree with values found in PLL0CON and PLL0CFG because changes to those registers do not take effect until a

proper PLL0 feed has occurred (see [Section 4.5.8 “PLL0 Feed register \(PLL0FEED - 0x400F C08C\)”](#)).

Table 22. PLL Status register (PLL0STAT - address 0x400F C088) bit description

Bit	Symbol	Description	Reset value
14:0	MSEL0	Read-back for the PLL0 Multiplier value. This is the value currently used by PLL0, and is one less than the actual multiplier.	0
15	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
23:16	NSEL0	Read-back for the PLL0 Pre-Divider value. This is the value currently used by PLL0, and is one less than the actual divider.	0
24	PLLE0_STAT	Read-back for the PLL0 Enable bit. This bit reflects the state of the PLEC0 bit in PLL0CON (see Table 19) after a valid PLL0 feed. When one, PLL0 is currently enabled. When zero, PLL0 is turned off. This bit is automatically cleared when Power-down mode is entered.	0
25	PLLC0_STAT	Read-back for the PLL0 Connect bit. This bit reflects the state of the PLLC0 bit in PLL0CON (see Table 19) after a valid PLL0 feed. When PLLC0 and PLLE0 are both one, PLL0 is connected as the clock source for the CPU. When either PLLC0 or PLLE0 is zero, PLL0 is bypassed. This bit is automatically cleared when Power-down mode is entered.	0
26	PLOCK0	Reflects the PLL0 Lock status. When zero, PLL0 is not locked. When one, PLL0 is locked onto the requested frequency. See text for details.	0
31:27	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

4.5.6 PLL0 Interrupt: PLOCK0

The PLOCK0 bit in the PLL0STAT register reflects the lock status of PLL0. When PLL0 is enabled, or parameters are changed, PLL0 requires some time to establish lock under the new conditions. PLOCK0 can be monitored to determine when PLL0 may be connected for use. The value of PLOCK0 may not be stable when the PLL reference frequency (F_{REF} , the frequency of REFCLK, which is equal to the PLL input frequency divided by the pre-divider value) is less than 100 kHz or greater than 20 MHz. In these cases, the PLL may be assumed to be stable after a start-up time has passed. This time is 500 μ s when FREF is greater than 400 kHz and 200 / FREF seconds when FREF is less than 400 kHz.

PLOCK0 is connected to the interrupt controller. This allows for software to turn on PLL0 and continue with other functions without having to wait for PLL0 to achieve lock. When the interrupt occurs, PLL0 may be connected, and the interrupt disabled. PLOCK0 appears as interrupt 32 in [Table 50](#). Note that PLOCK0 remains asserted whenever PLL0 is locked, so if the interrupt is used, the interrupt service routine must disable the PLOCK0 interrupt prior to exiting.

4.5.7 PLL0 Modes

The combinations of PLLE0 and PLLC0 are shown in [Table 23](#).

Table 23. PLL control bit combinations

PLLC0	PLLE0	PLL Function
0	0	PLL0 is turned off and disconnected. PLL0 outputs the unmodified clock input.
0	1	PLL0 is active, but not yet connected. PLL0 can be connected after PLOCK0 is asserted.
1	0	Same as 00 combination. This prevents the possibility of PLL0 being connected without also being enabled.
1	1	PLL0 is active and has been connected as the system clock source.

4.5.8 PLL0 Feed register (PLL0FEED - 0x400F C08C)

A correct feed sequence must be written to the PLL0FEED register in order for changes to the PLL0CON and PLL0CFG registers to take effect. The feed sequence is:

1. Write the value 0xAA to PLL0FEED.
2. Write the value 0x55 to PLL0FEED.

The two writes must be in the correct sequence, and there must be no other register access in the same address space (0x400F C000 to 0x400F FFFF) between them. Because of this, it may be necessary to disable interrupts for the duration of the PLL0 feed operation, if there is a possibility that an interrupt service routine could write to another register in that space. If either of the feed values is incorrect, or one of the previously mentioned conditions is not met, any changes to the PLL0CON or PLL0CFG register will not become effective.

Table 24. PLL Feed register (PLL0FEED - address 0x400F C08C) bit description

Bit	Symbol	Description	Reset value
7:0	PLL0FEED	The PLL0 feed sequence must be written to this register in order for PLL0 configuration and control register changes to take effect.	0x00
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

4.5.9 PLL0 and Power-down mode

Power-down mode automatically turns off and disconnects PLL0. Wake-up from Power-down mode does not automatically restore PLL0 settings, this must be done in software. Typically, a routine to activate PLL0, wait for lock, and then connect PLL0 can be called at the beginning of any interrupt service routine that might be called due to the wake-up. It is important not to attempt to restart PLL0 by simply feeding it when execution resumes after a wake-up from Power-down mode. This would enable and connect PLL0 at the same time, before PLL lock is established.

4.5.10 PLL0 frequency calculation

PLL0 equations use the following parameters:

Table 25. PLL frequency parameter

Parameter	Description
F_{IN}	the frequency of PLLCLKIN from the Clock Source Selection Multiplexer.
F_{CCO}	the frequency of the PLLCLK (output of the PLL Current Controlled Oscillator)
N	PLL0 Pre-divider value from the NSEL0 bits in the PLL0CFG register (PLL0CFG NSEL0 field + 1). N is an integer from 1 through 32.
M	PLL0 Multiplier value from the MSEL0 bits in the PLL0CFG register (PLL0CFG MSEL0 field + 1). Not all potential values are supported. See below.
F_{REF}	PLL internal reference frequency, F_{IN} divided by N.

The PLL0 output frequency (when PLL0 is both active and connected) is given by:

$$F_{CCO} = (2 \times M \times F_{IN}) / N$$

PLL inputs and settings must meet the following:

- F_{IN} is in the range of 32 kHz to 50 MHz.
- F_{CCO} is in the range of 275 MHz to 550 MHz.

The equation can be solved for other PLL parameters:

$$M = (F_{CCO} \times N) / (2 \times F_{IN})$$

$$N = (2 \times M \times F_{IN}) / F_{CCO}$$

$$F_{IN} = (F_{CCO} \times N) / (2 \times M)$$

Allowed values for M:

At higher oscillator frequencies, in the MHz range, values of M from 6 through 512 are allowed. This supports the entire useful range of both the main oscillator and the IRC.

For lower frequencies, specifically when the RTC is used to clock PLL0, a set of 65 additional M values have been selected for supporting baud rate generation, CAN operation, and obtaining integer MHz frequencies. These values are shown in [Table 26](#).

Table 26. Additional Multiplier Values for use with a Low Frequency Clock Input

Low Frequency PLL Multipliers				
4272	4395	4578	4725	4807
5127	5188	5400	5493	5859
6042	6075	6104	6409	6592
6750	6836	6866	6958	7050
7324	7425	7690	7813	7935
8057	8100	8545	8789	9155
9613	10254	10376	10986	11719
12085	12207	12817	13184	13672
13733	13916	14099	14420	14648
15381	15564	15625	15869	16113

Table 26. Additional Multiplier Values for use with a Low Frequency Clock Input

Low Frequency PLL Multipliers				
16479	17578	18127	18311	19226
19775	20508	20599	20874	21149
21973	23071	23438	23804	24170

4.5.11 Procedure for determining PLL0 settings

PLL0 parameter determination can be simplified by using a spreadsheet available from NXP. To determine PLL0 parameters by hand, the following general procedure may be used:

1. Determine if the application requires use of the USB interface, and whether it will be clocked from PLL0. The USB requires a 50% duty cycle clock of 48 MHz within a very small tolerance, which means that F_{CCO} must be an even integer multiple of 48 MHz (i.e. an integer multiple of 96 MHz), within a very small tolerance.
2. Choose the desired processor operating frequency (CCLK). This may be based on processor throughput requirements, need to support a specific set of UART baud rates, etc. Bear in mind that peripheral devices may be running from a lower clock frequency than that of the processor (see [Section 4.7 “Clock dividers” on page 56](#) and [Section 4.8 “Power control” on page 60](#)). Find a value for F_{CCO} that is close to a multiple of the desired CCLK frequency, bearing in mind the requirement for USB support in [1] above, and that lower values of F_{CCO} result in lower power dissipation.
3. Choose a value for the PLL input frequency (F_{IN}). This can be a clock obtained from the main oscillator, the RTC oscillator, or the on-chip RC oscillator. For USB support, the main oscillator should be used. Bear in mind that if PLL1 rather than PLL0 is used to clock the USB subsystem, this affects the choice of the main oscillator frequency.
4. Calculate values for M and N to produce a sufficiently accurate F_{CCO} frequency. The desired M value -1 will be written to the MSEL0 field in PLL0CFG. The desired N value -1 will be written to the NSEL0 field in PLL0CFG.

In general, it is better to use a smaller value for N, to reduce the level of multiplication that must be accomplished by the CCO. Due to the difficulty in finding the best values in some cases, it is recommended to use a spreadsheet or similar method to show many possibilities at once, from which an overall best choice may be selected. A spreadsheet is available from NXP for this purpose.

4.5.12 Examples of PLL0 settings

The following table gives a summary of examples that illustrate selecting PLL0 values based on different system requirements.

Table 27. Summary of PLL0 examples

Example	Description
1	- The PLL0 clock source is 10 MHz. - PLL0 is not used as the USB clock source, or the USB interface is not used. - The desired CPU clock is 100 MHz.
2	- The PLL0 clock source is 4 MHz. - PLL0 is used as the USB clock source. - The desired CPU clock is 60 MHz.
3	- The PLL0 clock source is the 32.768 kHz RTC clock. - PLL0 is not used as the USB clock source, or the USB interface is not used. - The desired CPU clock is 72 MHz.

Example 1

Assumptions:

- The USB interface will not be used in the application, or will be clocked by PLL1.
- The desired CPU rate is 100 MHz.
- An external 10 MHz crystal or clock source will be used as the system clock source.

Calculations:

$$M = (F_{CCO} \times N) / (2 \times F_{IN})$$

A smaller value for the PLL pre-divide (N) as well as a smaller value of the multiplier (M), both result in better PLL operational stability and lower output jitter. Lower values of F_{CCO} also save power. So, the process of determining PLL setup parameters involves looking for the smallest N and M values giving the lowest F_{CCO} value that will support the required CPU and/or USB clocks. It is usually easier to work backward from the desired output clock rate and determine a target F_{CCO} rate, then find a way to obtain that F_{CCO} rate from the available input clock.

Potential precise values of F_{CCO} are integer multiples of the desired CPU clock. In this example, it is clear that the smallest frequency for F_{CCO} that can produce the desired CPU clock rate and is within the PLL0 operating range of 275 to 550 MHz is 300 MHz (3×100 MHz).

Assuming that the PLL pre-divide is 1 ($N = 1$), the equation above gives $M = ((300 \times 10^6) / (2 \times 10^6)) = 300 / 2 = 15$. Since the result is an integer, there is no need to look any further for a good set of PLL0 configuration values. The value written to PLL0CFG would be 0x0E ($N - 1 = 0$; $M - 1 = 14$ gives 0x0E).

The PLL output must be further divided in order to produce the CPU clock. This is accomplished using a separate divider that is described later in this chapter, see [Section 4.7.1](#).

Example 2

Assumptions:

- The USB interface will be used in the application and will be clocked from PLL0.
- The desired CPU rate is 60 MHz.
- An external 4 MHz crystal or clock source will be used as the system clock source. This clock source could be the Internal RC oscillator (IRC).

Calculations:

$$M = (F_{CCO} \times N) / (2 \times F_{IN})$$

Because supporting USB requires a precise 48 MHz clock with a 50% duty cycle, that need must be addressed first. Potential precise values of F_{CCO} are integer multiples of the 2 × the 48 MHz USB clock. The 2 × insures that the clock has a 50% duty cycle, which would not be the case for a division of the PLL output by an odd number.

The possibilities for the F_{CCO} rate when the USB is used are 288 MHz, 384 MHz, and 480 MHz. The smallest frequency for F_{CCO} that can produce a valid USB clock rate and is within the PLL0 operating range is 288 MHz (3 × 2 × 48 MHz).

Start by assuming $N = 1$, since this produces the smallest multiplier needed for PLL0. So, $M = ((288 \times 10^6) \times 1) / (2 \times (4 \times 10^6)) = 288 / 8 = 36$. The result is an integer, which is necessary to obtain a precise USB clock. The value written to PLL0CFG would be 0x23 ($N - 1 = 0$; $M - 1 = 35 = 0x23$).

The potential CPU clock rate can be determined by dividing F_{CCO} by the desired CPU frequency: $288 \times 10^6 / 60 \times 10^6 = 4.8$. The nearest integer value for the CPU Clock Divider is then 5, giving us 57.6 MHz as the nearest value to the desired CPU clock rate.

If it is important to obtain exactly 60 MHz, an F_{CCO} rate must be found that can be divided down to both 48 MHz and 60 MHz. As previously noted, the possibilities for the F_{CCO} rate when the USB is used are 288 MHz, 384 MHz, and 480 MHz. Of these, only 480 MHz is also evenly divisible by 60. Divided by 10, this gives the 48 MHz with a 50% duty cycle needed by the USB subsystem. Divided by 8, it gives 60 MHz for the CPU clock. PLL0 settings for 480 MHz are $N = 1$ and $M = 60$.

The PLL output must be further divided in order to produce both the CPU clock and the USB clock. This is accomplished using separate dividers that are described later in this chapter. See [Section 4.7.1](#) and [Section 4.7.2](#).

Example 3

Assumptions:

- The USB interface will not be used in the application, or will be clocked by PLL1.
- The desired CPU rate is 72 MHz
- The 32.768 kHz RTC clock source will be used as the system clock source

Calculations:

$$M = (F_{CCO} \times N) / (2 \times F_{IN})$$

The smallest integer multiple of the desired CPU clock rate that is within the PLL0 operating range is 288 MHz (4×72 MHz).

Using the equation above and assuming that $N = 1$, $M = ((288 \times 10^6) / 1) / (2 \times 32,768) = 4,394.53125$. This is not an integer, so the CPU frequency will not be exactly 72 MHz with this setting. Since this example is less obvious, it may be useful to make a table of possibilities for different values of N (see below).

Table 28. Potential values for PLL example

N	M	M Rounded	FREF in Hz (F_{IN} / N)	FCCO in MHz ($F_{REF} \times M$)	CCLK in MHz ($F_{CCO} / 4$)	% Error ($(CCLK - 72) / 72$)
1	4394.53125	4395	32768	288.0307	72.0077	0.0107
2	8789.0625	8789	16384	287.9980	71.9995	-0.0007
3	13183.59375	13184	10922.67	288.0089	72.0022	0.0031
4	17578.125	17578	8192	287.9980	71.9995	-0.0007
5	21972.65625	21973	6553.6	288.0045	72.0011	0.0016

Beyond $N = 5$, the value of M is out of range or not supported, so the table stops at that point. In the third column of the table, the calculated M value is rounded to the nearest integer. If this results in CCLK being above the maximum operating frequency, it is allowed if it is not more than 1/2 % above the maximum frequency.

In general, larger values of FREF result in a more stable PLL when the input clock is a low frequency. Even the first table entry shows a very small error of just over 1 hundredth of a percent, or 107 parts per million (ppm). If that is not accurate enough in the application, the second case gives a much smaller error of 7 ppm. There are no allowed combinations that give a smaller error than that.

Remember that when a frequency below about 1 MHz is used as the PLL0 clock source, not all multiplier values are available. As it turns out, all of the rounded M values found in [Table 28](#) of this example are supported, which may be confirmed in [Table 26](#). If PLL0 calculations suggest use of unsupported multiplier values, those values must be disregarded and other values examined to find the best fit.

The value written to PLL0CFG for the second table entry would be 0x12254 ($N - 1 = 1 = 0x1$; $M - 1 = 8788 = 0x2254$).

The PLL output must be further divided in order to produce the CPU clock. This is accomplished using a separate divider that is described later in this chapter, see [Section 4.7.1](#).

4.5.13 PLL0 setup sequence

The following sequence must be followed step by step in order to have PLL0 initialized and running:

1. Disconnect PLL0 with one feed sequence if PLL0 is already connected.
2. Disable PLL0 with one feed sequence.
3. Change the CPU Clock Divider setting to speed up operation without PLL0, if desired.
4. Write to the Clock Source Selection Control register to change the clock source if needed.
5. Write to the PLL0CFG and make it effective with one feed sequence. The PLL0CFG can only be updated when PLL0 is disabled.
6. Enable PLL0 with one feed sequence.
7. Change the CPU Clock Divider setting for the operation with PLL0. It is critical to do this before connecting PLL0.
8. Wait for PLL0 to achieve lock by monitoring the PLOCK0 bit in the PLL0STAT register, or using the PLOCK0 interrupt, or wait for a fixed time when the input clock to PLL0 is slow (i.e. 32 kHz). The value of PLOCK0 may not be stable when the PLL reference frequency (FREF, the frequency of REFCLK, which is equal to the PLL input frequency divided by the pre-divider value) is less than 100 kHz or greater than 20 MHz. In these cases, the PLL may be assumed to be stable after a start-up time has passed. This time is 500 μ s when FREF is greater than 400 kHz and 200 / FREF seconds when FREF is less than 400 kHz.
9. Connect PLL0 with one feed sequence.

It is very important not to merge any steps above. For example, do not update the PLL0CFG and enable PLL0 simultaneously with the same feed sequence.

4.6 PLL1 (Phase Locked Loop 1)

PLL1 receives its clock input from the main oscillator only and can be used to provide a fixed 48 MHz clock only to the USB subsystem. This is an option in addition to the possibility of generating the USB clock from PLL0.

PLL1 is disabled and powered off on reset. If PLL1 is left disabled, the USB clock can be supplied by PLL0 if everything is set up to provide 48 MHz through that route. If PLL1 is enabled and connected via the PLL1CON register (see [Section 4.6.2](#)), it is automatically selected to drive the USB subsystem (see [Figure 7](#)).

PLL1 activation is controlled via the PLL1CON register. PLL1 multiplier and divider values are controlled by the PLL1CFG register. These two registers are protected in order to prevent accidental alteration of PLL1 parameters or deactivation of PLL1. The protection is accomplished by a feed sequence similar to that of the Watchdog Timer. Details are provided in the description of the PLL1FEED register.

PLL1 accepts an input clock frequency in the range of 10 MHz to 25 MHz only. The input frequency is multiplied up to the range of 48 MHz for the USB clock using a Current Controlled Oscillator (CCO). The multiplier can be an integer value from 1 to 32 (for USB, the multiplier value cannot be higher than 4). The CCO operates in the range of 156 MHz to 320 MHz, so there is an additional divider in the loop to keep the CCO within its frequency range while PLL1 is providing the desired output frequency. The output divider may be set to divide by 2, 4, 8, or 16 to produce the output clock. Since the minimum output divider value is 2, it is insured that the output of PLL1 has a 50% duty cycle. A block diagram of PLL1 is shown in [Figure 10](#).

4.6.1 PLL1 register description

PLL1 is controlled by the registers shown in [Table 29](#). More detailed descriptions follow. Writes to any unused bits are ignored. A read of any unused bits will return a logic zero.

Warning: Improper setting of PLL1 values may result in incorrect operation of the USB subsystem!

Table 29. PLL1 registers

Name	Description	Access	Reset value ^[1]	Address
PLL1CON	PLL1 Control Register. Holding register for updating PLL1 control bits. Values written to this register do not take effect until a valid PLL1 feed sequence has taken place.	R/W	0	0x400F C0A0

Table 29. PLL1 registers

Name	Description	Access	Reset value ^[1]	Address
PLL1CFG	PLL1 Configuration Register. Holding register for updating PLL1 configuration values. Values written to this register do not take effect until a valid PLL1 feed sequence has taken place.	R/W	0	0x400F C0A4
PLL1STAT	PLL1 Status Register. Read-back register for PLL1 control and configuration information. If PLL1CON or PLL1CFG have been written to, but a PLL1 feed sequence has not yet occurred, they will not reflect the current PLL1 state. Reading this register provides the actual values controlling PLL1, as well as PLL1 status.	RO	0	0x400F C0A8
PLL1FEED	PLL1 Feed Register. This register enables loading of PLL1 control and configuration information from the PLL1CON and PLL1CFG registers into the shadow registers that actually affect PLL1 operation.	WO	NA	0x400F C0AC

[1] Reset Value reflects the data stored in used bits only. It does not include reserved bits content.

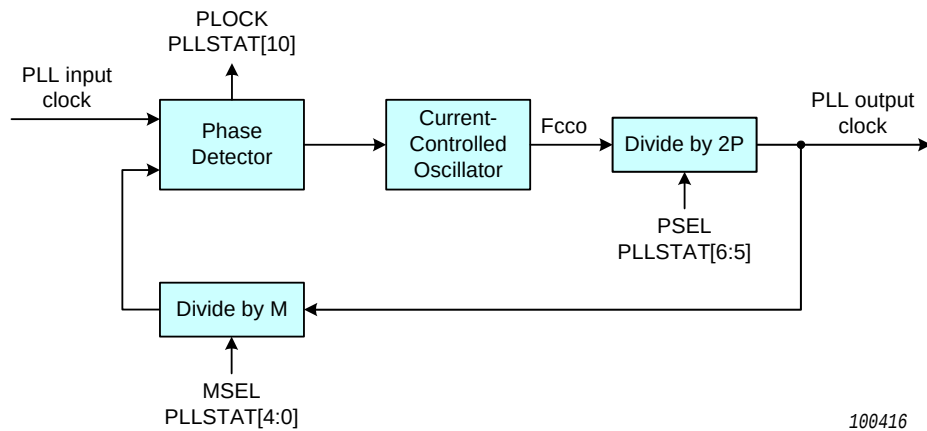


Fig 10. PLL1 block diagram

4.6.2 PLL1 Control register (PLL1CON - 0x400F C0A0)

The PLL1CON register contains the bits that enable and connect PLL1. Enabling PLL1 allows it to attempt to lock to the current settings of the multiplier and divider values. Connecting PLL1 causes the USB subsystem to run from the PLL1 output clock. Changes to the PLL1CON register do not take effect until a correct PLL feed sequence has been given (see [Section 4.6.6](#) and [Section 4.6.3](#)).

Table 30. PLL1 Control register (PLL1CON - address 0x400F C0A0) bit description

Bit	Symbol	Description	Reset value
0	PLLE1	PLL1 Enable. When one, and after a valid PLL1 feed, this bit will activate PLL1 and allow it to lock to the requested frequency. See PLL1STAT register, Table 32 .	0
1	PLLC1	PLL1 Connect. Setting PLLC to one after PLL1 has been enabled and locked, then followed by a valid PLL1 feed sequence causes PLL1 to become the clock source for the USB subsystem via the USB clock divider. See PLL1STAT register, Table 32 .	0
31:2	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

PLL1 must be set up, enabled, and lock established before it may be used as a clock source for the USB subsystem. The hardware does not insure that the PLL is locked before it is connected nor does it automatically disconnect the PLL if lock is lost during operation.

4.6.3 PLL1 Configuration register (PLL1CFG - 0x400F C0A4)

The PLL1CFG register contains the PLL1 multiplier and divider values. Changes to the PLL1CFG register do not take effect until a correct PLL1 feed sequence has been given (see [Section 4.6.6](#)). Calculations for the PLL1 frequency, and multiplier and divider values are found in [Section 4.6.9](#).

Table 31. PLL Configuration register (PLL1CFG - address 0x400F C0A4) bit description

Bit	Symbol	Description	Reset value
4:0	MSEL1	PLL1 Multiplier value. Supplies the value "M" in the PLL1 frequency calculations. Note: For details on selecting the right value for MSEL1 see Section 4.6.8 .	0
6:5	PSEL1	PLL1 Divider value. Supplies the value "P" in the PLL1 frequency calculations. Note: For details on selecting the right value for PSEL1 see Section 4.6.8 .	0
31:7	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

4.6.4 PLL1 Status register (PLL1STAT - 0x400F C0A8)

The read-only PLL1STAT register provides the actual PLL1 parameters that are in effect at the time it is read, as well as the PLL1 status. PLL1STAT may disagree with values found in PLL1CON and PLL1CFG because changes to those registers do not take effect until a proper PLL1 feed has occurred (see [Section 4.6.6 "PLL1 Feed register \(PLL1FEED - 0x400F C0AC\)"](#)).

Table 32. PLL1 Status register (PLL1STAT - address 0x400F C0A8) bit description

Bit	Symbol	Description	Reset value
4:0	MSEL1	Read-back for the PLL1 Multiplier value. This is the value currently used by PLL1.	0
6:5	PSEL1	Read-back for the PLL1 Divider value. This is the value currently used by PLL1.	0
7	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
8	PLLE1_STAT	Read-back for the PLL1 Enable bit. When one, PLL1 is currently activated. When zero, PLL1 is turned off. This bit is automatically cleared when Power-down mode is activated.	0
9	PLLC1_STAT	Read-back for the PLL1 Connect bit. When PLLC and PLLE are both one, PLL1 is connected as the clock source for the microcontroller. When either PLLC or PLLE is zero, PLL1 is bypassed and the oscillator clock is used directly by the microcontroller. This bit is automatically cleared when Power-down mode is activated.	0
10	PLOCK1	Reflects the PLL1 Lock status. When zero, PLL1 is not locked. When one, PLL1 is locked onto the requested frequency.	0
31:11	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

4.6.4.1 PLL1 modes

The combinations of PLLE1 and PLLC1 are shown in [Table 33](#).

Table 33. PLL1 control bit combinations

PLLC1	PLLE1	PLL1 Function
0	0	PLL1 is turned off and disconnected.
0	1	PLL1 is active, but not yet connected. PLL1 can be connected after PLOCK1 is asserted.
1	0	Same as 00 combination. This prevents the possibility of PLL1 being connected without also being enabled.
1	1	PLL1 is active and has been connected. The clock for the USB subsystem is sourced from PLL1.

4.6.5 PLL1 Interrupt: PLOCK1

The PLOCK1 bit in the PLL1STAT register reflects the lock status of PLL1. When PLL1 is enabled, or parameters are changed, the PLL requires some time to establish lock under the new conditions. PLOCK1 can be monitored to determine when the PLL may be connected for use.

PLOCK1 is connected to the interrupt controller. This allows for software to turn on the PLL and continue with other functions without having to wait for the PLL to achieve lock. When the interrupt occurs, the PLL may be connected, and the interrupt disabled. PLOCK1 appears as interrupt 48 in [Table 50](#). Note that PLOCK1 remains asserted whenever PLL1 is locked, so if the interrupt is used, the interrupt service routine must disable the PLOCK1 interrupt prior to exiting.

4.6.6 PLL1 Feed register (PLL1FEED - 0x400F C0AC)

A correct feed sequence must be written to the PLL1FEED register in order for changes to the PLL1CON and PLL1CFG registers to take effect. The feed sequence is:

1. Write the value 0xAA to PLL1FEED.
2. Write the value 0x55 to PLL1FEED.

The two writes must be in the correct sequence, and there must be no other register access in the same address space (0x400F C000 to 0x400F FFFF) between them. Because of this, it may be necessary to disable interrupts for the duration of the PLL feed operation, if there is a possibility that an interrupt service routine could write to another register in that space. If either of the feed values is incorrect, or one of the previously mentioned conditions is not met, any changes to the PLL1CON or PLL1CFG register will not become effective.

Table 34. PLL1 Feed register (PLL1FEED - address 0x400F C0AC) bit description

Bit	Symbol	Description	Reset value
7:0	PLL1FEED	The PLL1 feed sequence must be written to this register in order for PLL1 configuration and control register changes to take effect.	0x00
31:8	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

4.6.7 PLL1 and Power-down mode

Power-down mode automatically turns off and disconnects activated PLL(s). Wake-up from Power-down mode does not automatically restore PLL settings, this must be done in software. Typically, a routine to activate the PLL, wait for lock, and then connect the PLL can be called at the beginning of any interrupt service routine that might be called due to the wake-up. It is important not to attempt to restart a PLL by simply feeding it when execution resumes after a wake-up from Power-down mode. This would enable and connect the PLL at the same time, before PLL lock is established.

If activity on the USB data lines is not selected to wake the microcontroller from Power-down mode (see [Section 4.8.8](#) for details of wake up from reduced modes), both the Main PLL (PLL0) and the USB PLL (PLL1) will be automatically be turned off and disconnected when Power-down mode is invoked, as described above. However, if the USB activity interrupt is enabled and USB_NEED_CLK = 1 (see [Table 192](#) for a description of USB_NEED_CLK), it is not possible to go into Power-down mode and any attempt to set the PD bit will fail, leaving the PLLs in the current state.

4.6.8 PLL1 frequency calculation

The PLL1 equations use the following parameters:

Table 35. Elements determining PLL frequency

Element	Description
F _{OSC}	the frequency from the crystal oscillator
F _{CCO}	the frequency of the PLL1 current controlled oscillator
USBCLK	the PLL1 output frequency (48 MHz for USB)
M	PLL1 Multiplier value from the MSEL1 bits in the PLL1CFG register
P	PLL1 Divider value from the PSEL1 bits in the PLL1CFG register

The PLL1 output frequency (when the PLL is both active and connected) is given by:

$$\text{USBCLK} = M \times F_{\text{OSC}} \text{ or } \text{USBCLK} = F_{\text{CCO}} / (2 \times P)$$

The CCO frequency can be computed as:

$$F_{\text{CCO}} = \text{USBCLK} \times 2 \times P \text{ or } F_{\text{CCO}} = F_{\text{OSC}} \times M \times 2 \times P$$

The PLL1 inputs and settings must meet the following criteria:

- F_{OSC} is in the range of 10 MHz to 25 MHz.
- USBCLK is 48 MHz.
- F_{CCO} is in the range of 156 MHz to 320 MHz.

4.6.9 Procedure for determining PLL1 settings

The PLL1 configuration for USB may be determined as follows:

1. The desired PLL1 output frequency is USBCLK = 48 MHz.
2. Choose an oscillator frequency (F_{OSC}). USBCLK must be the whole (non-fractional) multiple of F_{OSC} meaning that the possible values for F_{OSC} are 12 MHz, 16 MHz, and 24 MHz.
3. Calculate the value of M to configure the MSEL1 bits. $M = \text{USBCLK} / F_{\text{OSC}}$. In this case, the possible values for M = 2, 3, or 4 (F_{OSC} = 24 MHz, 16 MHz, or 12 MHz). The value written to the MSEL1 bits in PLL1CFG is M - 1 (see [Table 37](#)).
4. Find a value for P to configure the PSEL1 bits, such that F_{CCO} is within its defined frequency limits of 156 MHz to 320 MHz. F_{CCO} is calculated using $F_{\text{CCO}} = \text{USBCLK} \times 2 \times P$. It follows that P = 2 is the only P value to yield F_{CCO} in the allowed range. The value written to the PSEL1 bits in PLL1CFG is '01' for P = 2 (see [Table 36](#)).

Table 36. PLL1 Divider values*Values allowed for using PLL1 with USB are highlighted.*

PSEL1 Bits (PLL1CFG bits [6:5])	Value of P
00	1
01	2
10	4
11	8

Table 37. PLL1 Multiplier values*Values allowed for using PLL1 with USB are highlighted.*

MSEL1 Bits (PLL1CFG bits [4:0])	Value of M
00000	1
00001	2
00010	3
00011	4
...	...
11110	31
11111	32

4.7 Clock dividers

The output of the PLL0 must be divided down for use by the CPU and the USB subsystem (if used with PLL0, see [Section 4.6](#)). Separate dividers are provided such that the CPU frequency can be determined independently from the USB subsystem, which always requires 48 MHz with a 50% duty cycle for proper operation.

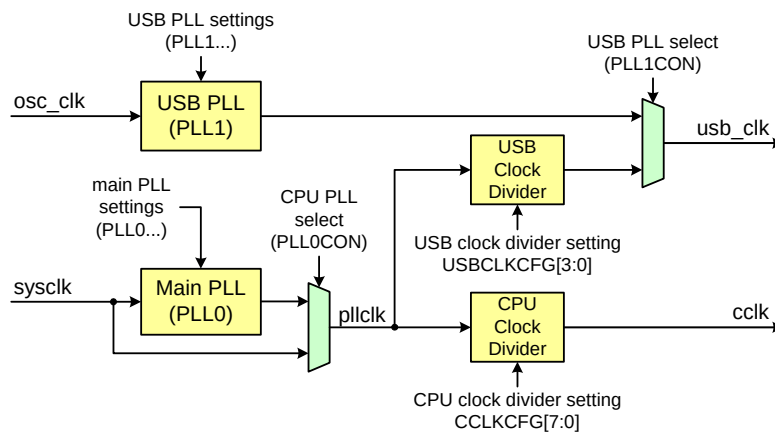


Fig 11. PLLs and clock dividers

4.7.1 CPU Clock Configuration register (CCLKCFG - 0x400F C104)

The CCLKCFG register controls the division of the PLL0 output before it is used by the CPU. When PLL0 is bypassed, the division may be by 1. When PLL0 is running, the output must be divided in order to bring the CPU clock frequency (CCLK) within operating limits. An 8-bit divider allows a range of options, including slowing CPU operation to a low rate for temporary power savings without turning off PLL0.

Table 38. CPU Clock Configuration register (CCLKCFG - address 0x400F C104) bit description

Bit	Symbol	Value	Description	Reset value
7:0	CCLKSEL		Selects the divide value for creating the CPU clock (CCLK) from the PLL0 output.	0x00
		0	pllclk is divided by 1 to produce the CPU clock. This setting is not allowed when the PLL0 is connected, because the rate would always be greater than the maximum allowed CPU clock.	
		1	pllclk is divided by 2 to produce the CPU clock. This setting is not allowed when the PLL0 is connected, because the rate would always be greater than the maximum allowed CPU clock.	
		2	pllclk is divided by 3 to produce the CPU clock.	
		3	pllclk is divided by 4 to produce the CPU clock.	
		4	pllclk is divided by 5 to produce the CPU clock.	
		:	:	
		255	pllclk is divided by 256 to produce the CPU clock.	
31:8	-		Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

The CCLK is derived from the PLL0 output signal, divided by CCLKSEL + 1. Having CCLKSEL = 2 results in CCLK being one third of the PLL0 output, CCLKSEL = 3 results in CCLK being one quarter of the PLL0 output, etc.

4.7.2 USB Clock Configuration register (USBCLKCFG - 0x400F C108)

This register is used only if the USB PLL (PLL1) is not connected (via the PLLC1 bit in PLL1CON). If PLL1 is connected, its output is automatically used as the USB clock source, and PLL1 must be configured to supply the correct 48 MHz clock to the USB subsystem. If PLL1 is not connected, the USB subsystem will be driven by PLL0 via the USB clock divider.

The USBCLKCFG register controls the division of the PLL0 output before it is used by the USB subsystem. The PLL0 output must be divided in order to bring the USB clock frequency to 48 MHz with a 50% duty cycle. A 4-bit divider allows obtaining the correct USB clock from any even multiple of 48 MHz (i.e. any multiple of 96 MHz) within the PLL operating range.

Remark: The Internal RC oscillator should not be used to drive PLL0 when the USB is using PLL0 as a clock source because a more precise clock is needed for USB specification compliance (see [Table 17](#)).

Table 39. USB Clock Configuration register (USBCLKCFG - address 0x400F C108) bit description

Bit	Symbol	Value	Description	Reset value
3:0	USBSEL		Selects the divide value for creating the USB clock from the PLL0 output. Only the values shown below can produce even number multiples of 48 MHz from the PLL0 output. Warning: Improper setting of this value will result in incorrect operation of the USB interface.	0
		5	PLL0 output is divided by 6. PLL0 output must be 288 MHz.	
		7	PLL0 output is divided by 8. PLL0 output must be 384 MHz.	
		9	PLL0 output is divided by 10. PLL0 output must be 480 MHz.	
31:4	-		Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

4.7.3 Peripheral Clock Selection registers 0 and 1 (PCLKSEL0 - 0x400F C1A8 and PCLKSEL1 - 0x400F C1AC)

A pair of bits in a Peripheral Clock Selection register controls the rate of the clock signal that will be supplied to the corresponding peripheral as specified in [Table 40](#), [Table 41](#) and [Table 42](#).

Remark: The peripheral clock for the RTC block is fixed at CCLK/8.

Table 40. Peripheral Clock Selection register 0 (PCLKSEL0 - address 0x400F C1A8) bit description

Bit	Symbol	Description	Reset value
1:0	PCLK_WDT	Peripheral clock selection for WDT.	00
3:2	PCLK_TIMER0	Peripheral clock selection for TIMER0.	00
5:4	PCLK_TIMER1	Peripheral clock selection for TIMER1.	00
7:6	PCLK_UART0	Peripheral clock selection for UART0.	00
9:8	PCLK_UART1	Peripheral clock selection for UART1.	00
11:10	-	Reserved.	NA
13:12	PCLK_PWM1	Peripheral clock selection for PWM1.	00
15:14	PCLK_I2C0	Peripheral clock selection for I2C0.	00
17:16	PCLK_SPI	Peripheral clock selection for SPI.	00
19:18	-	Reserved.	NA
21:20	PCLK_SSP1	Peripheral clock selection for SSP1.	00
23:22	PCLK_DAC	Peripheral clock selection for DAC.	00
25:24	PCLK_ADC	Peripheral clock selection for ADC.	00
27:26	PCLK_CAN1	Peripheral clock selection for CAN1. [1]	00
29:28	PCLK_CAN2	Peripheral clock selection for CAN2. [1]	00
31:30	PCLK_ACF	Peripheral clock selection for CAN acceptance filtering. [1]	00

[1] PCLK_CAN1 and PCLK_CAN2 must have the same PCLK divide value when the CAN function is used.

Table 41. Peripheral Clock Selection register 1 (PCLKSEL1 - address 0x400F C1AC) bit description

Bit	Symbol	Description	Reset value
1:0	PCLK_QEI	Peripheral clock selection for the Quadrature Encoder Interface.	00
3:2	PCLK_GPIOINT	Peripheral clock selection for GPIO interrupts.	00
5:4	PCLK_PCB	Peripheral clock selection for the Pin Connect block.	00
7:6	PCLK_I2C1	Peripheral clock selection for I2C1.	00
9:8	-	Reserved.	NA
11:10	PCLK_SSP0	Peripheral clock selection for SSP0.	00
13:12	PCLK_TIMER2	Peripheral clock selection for TIMER2.	00
15:14	PCLK_TIMER3	Peripheral clock selection for TIMER3.	00
17:16	PCLK_UART2	Peripheral clock selection for UART2.	00
19:18	PCLK_UART3	Peripheral clock selection for UART3.	00
21:20	PCLK_I2C2	Peripheral clock selection for I2C2.	00
23:22	PCLK_I2S	Peripheral clock selection for I2S.	00
25:24	-	Reserved.	NA
27:26	PCLK_RIT	Peripheral clock selection for Repetitive Interrupt Timer.	00
29:28	PCLK_SYSCON	Peripheral clock selection for the System Control block.	00
31:30	PCLK_MC	Peripheral clock selection for the Motor Control PWM.	00

Table 42. Peripheral Clock Selection register bit values

PCLKSEL0 and PCLKSEL1 individual peripheral's clock select options	Function	Reset value
00	PCLK_peripheral = CCLK/4	00
01	PCLK_peripheral = CCLK	
10	PCLK_peripheral = CCLK/2	
11	PCLK_peripheral = CCLK/8, except for CAN1, CAN2, and CAN filtering when "11" selects = CCLK/6.	

4.8 Power control

The LPC176x/5x supports a variety of power control features: Sleep mode, Deep Sleep mode, Power-down mode, and Deep Power-down mode. The CPU clock rate may also be controlled as needed by changing clock sources, re-configuring PLL values, and/or altering the CPU clock divider value. This allows a trade-off of power versus processing speed based on application requirements. In addition, Peripheral Power Control allows shutting down the clocks to individual on-chip peripherals, allowing fine tuning of power consumption by eliminating all dynamic power use in any peripherals that are not required for the application.

Entry to any reduced power mode begins with the execution of either a WFI (Wait For Interrupt) or WFE (Wait For Exception) instruction by the Cortex-M3. The Cortex-M3 internally supports two reduced power modes: Sleep and Deep Sleep. These are selected by the SLEEPDEEP bit in the cortex-M3 System Control Register. Power-down and Deep Power-down modes are selected by bits in the PCON register. See [Table 44](#). The same register contains flags that indicate whether entry into each reduced power mode actually occurred.

The LPC176x/5x also implements a separate power domain in order to allow turning off power to the bulk of the device while maintaining operation of the Real Time Clock.

Reduced power modes have some limitation during debug, see [Section 33.5](#) for more information.

4.8.1 Sleep mode

Note: Sleep mode on the LPC176x/5x corresponds to the Idle mode on LPC2xxx series devices. The name is changed because ARM has incorporated portions of reduced power mode control into the Cortex-M3. LPC176x/5x documentation uses the Cortex-M3 terminology where applicable.

When Sleep mode is entered, the clock to the core is stopped, and the SMFLAG bit in PCON is set, see [Table 44](#). Resumption from the Sleep mode does not need any special sequence but re-enabling the clock to the ARM core.

In Sleep mode, execution of instructions is suspended until either a Reset or an interrupt occurs. Peripheral functions continue operation during Sleep mode and may generate interrupts to cause the processor to resume execution. Sleep mode eliminates dynamic power used by the processor itself, memory systems and related controllers, and internal buses.

The GPDMA may operate in Sleep mode to access AHB SRAMs and peripherals with GPDMA support, but the GPDMA cannot access the flash memory or the main SRAM, which are disabled in order to save power.

Wake-up from Sleep mode will occur whenever any enabled interrupt occurs.

4.8.2 Deep Sleep mode

Note: Deep Sleep mode on the LPC176x/5x corresponds to the Sleep mode on LPC23xx and LPC24xx series devices. The name is changed because ARM has incorporated portions of reduced power mode control into the Cortex-M3. LPC176x/5x documentation uses the Cortex-M3 terminology where applicable.

When the chip enters the Deep Sleep mode, the main oscillator is powered down, nearly all clocks are stopped, and the DSFLAG bit in PCON is set, see [Table 44](#). The IRC remains running and can be configured to drive the Watchdog Timer, allowing the Watchdog to wake up the CPU. The 32 kHz RTC oscillator is not stopped and RTC interrupts may be used as a wake-up source. The flash is left in the standby mode allowing a quick wake-up. The PLLs are automatically turned off and disconnected. The CCLK and USBCLK clock dividers automatically get reset to zero.

The processor state and registers, peripheral registers, and internal SRAM values are preserved throughout Deep Sleep mode and the logic levels of chip pins remain static. The Deep Sleep mode can be terminated and normal operation resumed by either a Reset or certain specific interrupts that are able to function without clocks. Since all dynamic operation of the chip is suspended, Deep Sleep mode reduces chip power consumption to a very low value.

On the wake-up of Deep Sleep mode, if the IRC was used before entering Deep Sleep mode, a 2-bit IRC timer starts counting and the code execution and peripherals activities will resume after the timer expires (4 cycles). If the main external oscillator was used, the 12-bit main oscillator timer starts counting and the code execution will resume when the timer expires (4096 cycles). The user must remember to re-configure any required PLLs and clock dividers after the wake-up.

Wake-up from Deep Sleep mode can be brought about by NMI, External Interrupts EINT0 through EINT3, GPIO interrupts, the Ethernet Wake-on-LAN interrupt, Brownout Detect, an RTC Alarm interrupt, a Watchdog Timer time out, a USB input pin transition (USB activity interrupt), or a CAN input pin transition, when the related interrupt is enabled. Wake-up will occur whenever any enabled interrupt occurs.

4.8.3 Power-down mode

Power-down mode does everything that Deep Sleep mode does, but also turns off the flash memory. Entry to Power-down mode causes the PDFLAG bit in PCON to be set, see [Table 44](#). This saves more power, but requires waiting for resumption of flash operation before execution of code or data access in the flash memory can be accomplished.

When the chip enters Power-down mode, the IRC, the main oscillator, and all clocks are stopped. The RTC remains running if it has been enabled and RTC interrupts may be used to wake up the CPU. The flash is forced into Power-down mode. The PLLs are automatically turned off and disconnected. The CCLK and USBCLK clock dividers automatically get reset to zero.

Upon wake-up from Power-down mode, if the IRC was used before entering Power-down mode, after IRC-start-up time (about 60 μ s), the 2-bit IRC timer starts counting and expiring in 4 cycles. Code execution can then be resumed immediately following the expiration of the IRC timer if the code was running from SRAM. In the meantime, the flash wake-up timer measures flash start-up time of about 100 μ s. When it times out, access to the flash is enabled. The user must remember to re-configure any required PLLs and clock dividers after the wake-up.

Wake-up from Power-down mode can be brought about by NMI, External Interrupts EINT0 through EINT3, GPIO interrupts, the Ethernet Wake-on-LAN interrupt, Brownout Detect, an RTC Alarm interrupt, a USB input pin transition (USB activity interrupt), or a CAN input pin transition, when the related interrupt is enabled.

4.8.4 Deep Power-down mode

In Deep Power-down mode, power is shut off to the entire chip with the exception of the Real-Time Clock, the RESET pin, the WIC, and the RTC backup registers. Entry to Deep Power-down mode causes the DPDFLAG bit in PCON to be set, see [Table 44](#).

To optimize power conservation, the user has the additional option of turning off or retaining power to the 32 kHz oscillator. It is also possible to use external circuitry to turn off power to the on-chip regulator via the VDD(REG)(3V3) pins after entering Deep Power-down mode. Power to the on-chip regulator must be restored before device operation can be restarted.

Wake-up from Deep Power-down mode will occur when an external reset signal is applied, or the RTC interrupt is enabled and an RTC interrupt is generated.

4.8.5 Peripheral power control

A Power Control for Peripherals feature allows individual peripherals to be turned off if they are not needed in the application, resulting in additional power savings. This is detailed in the description of the PCONP register.

4.8.6 Register description

The Power Control function uses registers shown in [Table 43](#). More detailed descriptions follow.

Table 43. Power Control registers

Name	Description	Access	Reset value ^[1]	Address
PCON	Power Control Register. This register contains control bits that enable some reduced power operating modes of the LPC176x/5x. See Table 44 .	R/W	0x00	0x400F C0C0
PCONP	Power Control for Peripherals Register. This register contains control bits that enable and disable individual peripheral functions, allowing elimination of power consumption by peripherals that are not needed.	R/W		0x400F C0C4

[1] Reset Value reflects the data stored in used bits only. It does not include reserved bits content.

4.8.7 Power Mode Control register (PCON - 0x400F C0C0)

Controls for some reduced power modes and other power related controls are contained in the PCON register, as described in [Table 44](#).

Table 44. Power Mode Control register (PCON - address 0x400F C0C0) bit description

Bit	Symbol	Description	Reset value
0	PM0	Power mode control bit 0. This bit controls entry to the Power-down mode. See Section 4.8.7.1 below for details.	0
1	PM1	Power mode control bit 1. This bit controls entry to the Deep Power-down mode. See Section 4.8.7.1 below for details.	0
2	BODRPM	Brown-Out Reduced Power Mode. When BODRPM is 1, the Brown-Out Detect circuitry will be turned off when chip Power-down mode or Deep Sleep mode is entered, resulting in a further reduction in power usage. However, the possibility of using Brown-Out Detect as a wake-up source from the reduced power mode will be lost. When 0, the Brown-Out Detect function remains active during Power-down and Deep Sleep modes. See the System Control Block chapter for details of Brown-Out detection.	0
3	BOGD ^[1]	Brown-Out Global Disable. When BOGD is 1, the Brown-Out Detect circuitry is fully disabled at all times, and does not consume power. When 0, the Brown-Out Detect circuitry is enabled. See the System Control Block chapter for details of Brown-Out detection. Note: the Brown-Out Reset Disable (BORD, in this register) and the Brown-Out Interrupt (xx) must be disabled when software changes the value of this bit.	0
4	BORD	Brown-Out Reset Disable. When BORD is 1, the BOD will not reset the device when the $V_{DD(REG)(3V3)}$ voltage dips goes below the BOD reset trip level. The Brown-Out interrupt is not affected. When BORD is 0, the BOD reset is enabled. See the Section 3.5 for details of Brown-Out detection.	0
7:5	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA
8	SMFLAG	Sleep Mode entry flag. Set when the Sleep mode is successfully entered. Cleared by software writing a one to this bit.	0 ^{[2][3]}
9	DSFLAG	Deep Sleep entry flag. Set when the Deep Sleep mode is successfully entered. Cleared by software writing a one to this bit.	0 ^{[2][3]}
10	PDFLAG	Power-down entry flag. Set when the Power-down mode is successfully entered. Cleared by software writing a one to this bit.	0 ^{[2][3]}
11	DPDFLAG	Deep Power-down entry flag. Set when the Deep Power-down mode is successfully entered. Cleared by software writing a one to this bit.	0 ^{[2][4]}
31:12	-	Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA

[1] BOD reset (BORD- bit 4 in PCON register) and BOD interrupt needs to be disabled before a user disables or enables the power to BOD (BOGD - bit 3 in PCON register).

[2] Only one of these flags will be valid at a specific time.

[3] Hardware reset only for a power-up of core power or by a brownout detect event.

[4] Hardware reset only for a power-up event on Vbat.

4.8.7.1 Encoding of Reduced Power Modes

The PM1 and PM0 bits in PCON allow entering reduced power modes as needed. The encoding of these bits allows backward compatibility with devices that previously only supported Sleep and Power-down modes. [Table 45](#) below shows the encoding for the three reduced power modes supported by the LPC176x/5x.

Table 45. Encoding of reduced power modes

PM1, PM0	Description
00	Execution of WFI or WFE enters either Sleep or Deep Sleep mode as defined by the SLEEPDEEP bit in the Cortex-M3 System Control Register.
01	Execution of WFI or WFE enters Power-down mode if the SLEEPDEEP bit in the Cortex-M3 System Control Register is 1.
10	Reserved, this setting should not be used.
11	Execution of WFI or WFE enters Deep Power-down mode if the SLEEPDEEP bit in the Cortex-M3 System Control Register is 1.

4.8.8 Wake-up from Reduced Power Modes

Any enabled interrupt can wake up the CPU from Sleep mode. Certain interrupts can wake up the processor if it is in either Deep Sleep mode or Power-down mode.

Interrupts that can occur during Deep Sleep or Power-down mode will wake up the CPU if the interrupt is enabled. After wake-up, execution will continue to the appropriate interrupt service routine. These interrupts are NMI, External Interrupts EINT0 through EINT3, GPIO interrupts, Ethernet Wake-on-LAN interrupt, Brownout Detect, RTC Alarm, CAN Activity Interrupt, and USB Activity Interrupt. In addition, the watchdog timer can wake up the part from Deep Sleep mode if the watchdog timer is being clocked by the IRC oscillator. For the wake-up process to take place the corresponding interrupt must be enabled in the NVIC. For pin-related peripheral functions, the related functions must also be mapped to pins.

The CAN Activity Interrupt is generated by activity on the CAN bus pins, and the USB Activity Interrupt is generated by activity on the USB bus pins. These interrupts are only useful to wake up the CPU when it is on Deep Sleep or Power-down mode, when the peripheral functions are powered up, but not active. Typically, if these interrupts are used, their flags should be polled just before enabling the interrupt and entering the desired reduced power mode. This can save time and power by avoiding an immediate wake-up. Upon wake-up, the interrupt service can turn off the related activity interrupt, do any application specific setup, and exit to await a normal peripheral interrupt.

In Deep Power-down mode, internal power to most of the device is removed, which limits the possibilities for waking up from this mode. Wake-up from Deep Power-down mode will occur when an external reset signal is applied, or the RTC interrupt is enabled and an RTC interrupt is generated.

4.8.9 Power Control for Peripherals register (PCONP - 0x400F C0C4)

The PCONP register allows turning off selected peripheral functions for the purpose of saving power. This is accomplished by gating off the clock source to the specified peripheral blocks. A few peripheral functions cannot be turned off (i.e. the Watchdog timer, the Pin Connect block, and the System Control block).

Some peripherals, particularly those that include analog functions, may consume power that is not clock dependent. These peripherals may contain a separate disable control that turns off additional circuitry to reduce power. Information on peripheral specific power saving features may be found in the chapter describing that peripheral.

Each bit in PCONP controls one peripheral as shown in [Table 46](#).

If a peripheral control bit is 1, that peripheral is enabled. If a peripheral control bit is 0, that peripheral's clock is disabled (gated off) to conserve power. For example if bit 19 is 1, the I²C1 interface is enabled. If bit 19 is 0, the I²C1 interface is disabled.

Important: valid read from a peripheral register and valid write to a peripheral register is possible only if that peripheral is enabled in the PCONP register!

Table 46. Power Control for Peripherals register (PCONP - address 0x400F C0C4) bit description

Bit	Symbol	Description	Reset value
0	-	Reserved.	NA
1	PCTIM0	Timer/Counter 0 power/clock control bit.	1
2	PCTIM1	Timer/Counter 1 power/clock control bit.	1
3	PCUART0	UART0 power/clock control bit.	1
4	PCUART1	UART1 power/clock control bit.	1
5	-	Reserved.	NA
6	PCPWM1	PWM1 power/clock control bit.	1
7	PCI2C0	The I ² C0 interface power/clock control bit.	1
8	PCSPI	The SPI interface power/clock control bit.	1
9	PCRTC	The RTC power/clock control bit.	1
10	PCSSP1	The SSP 1 interface power/clock control bit.	1
11	-	Reserved.	NA
12	PCADC	A/D converter (ADC) power/clock control bit. Note: Clear the PDN bit in the AD0CR before clearing this bit, and set this bit before setting PDN.	0
13	PCCAN1	CAN Controller 1 power/clock control bit.	0
14	PCCAN2	CAN Controller 2 power/clock control bit.	0
15	PCGPIO	Power/clock control bit for IOCON, GPIO, and GPIO interrupts.	1
16	PCRIT	Repetitive Interrupt Timer power/clock control bit.	0
17	PCMCPWM	Motor Control PWM	0
18	PCQEI	Quadrature Encoder Interface power/clock control bit.	0
19	PCI2C1	The I ² C1 interface power/clock control bit.	1
20	-	Reserved.	NA
21	PCSSP0	The SSP0 interface power/clock control bit.	1
22	PCTIM2	Timer 2 power/clock control bit.	0
23	PCTIM3	Timer 3 power/clock control bit.	0
24	PCUART2	UART 2 power/clock control bit.	0
25	PCUART3	UART 3 power/clock control bit.	0
26	PCI2C2	I2C interface 2 power/clock control bit.	1

Table 46. Power Control for Peripherals register (PCONP - address 0x400F C0C4) bit description

Bit	Symbol	Description	Reset value
27	PCI2S	I2S interface power/clock control bit.	0
28	-	Reserved.	NA
29	PCGPDMA	GPDMA function power/clock control bit.	0
30	PCENET	Ethernet block power/clock control bit.	0
31	PCUSB	USB interface power/clock control bit.	0

Note that the DAC peripheral does not have a control bit in PCONP. To enable the DAC, its output must be selected to appear on the related pin, P0.26, by configuring the PINSEL1 register. See [Section 8.5.2 "Pin Function Select Register 1 \(PINSEL1 - 0x4002 C004\)"](#).

4.8.10 Power control usage notes

After every reset, the PCONP register contains the value that enables selected interfaces and peripherals controlled by the PCONP to be enabled. Therefore, apart from proper configuring via peripheral dedicated registers, the user's application might have to access the PCONP in order to start using some of the on-board peripherals.

Power saving oriented systems should have 1s in the PCONP register only in positions that match peripherals really used in the application. All other bits, declared to be "Reserved" or dedicated to the peripherals not used in the current application, must be cleared to 0.

4.8.11 Power domains

The LPC176x/5x provides two independent power domains that allow the bulk of the device to have power removed while maintaining operation of the Real Time Clock.

The VBAT pin supplies power only to the RTC domain. The RTC requires a minimum of power to operate, which can be supplied by an external battery. Whenever the device core power is present, that power is used to operate the RTC, causing no power drain from a battery when main power is available.

4.9 Wake-up timer

The LPC176x/5x begins operation at power-up and when awakened from Power-down mode by using the 4 MHz IRC oscillator as the clock source. This allows chip operation to begin quickly. If the main oscillator or one or both PLLs are needed by the application, software will need to enable these features and wait for them to stabilize before they are used as a clock source.

When the main oscillator is initially activated, the wake-up timer allows software to ensure that the main oscillator is fully functional before the processor uses it as a clock source and starts to execute instructions. This is important at power-on, all types of Reset, and whenever any of the aforementioned functions are turned off for any reason. Since the oscillator and other functions are turned off during Power-down mode, any wake-up of the processor from Power-down mode makes use of the Wake-up Timer.

The Wake-up Timer monitors the crystal oscillator as the means of checking whether it is safe to begin code execution. When power is applied to the chip, or some event caused the chip to exit Power-down mode, some time is required for the oscillator to produce a signal of sufficient amplitude to drive the clock logic. The amount of time depends on many factors, including the rate of $V_{DD(REG)(3V3)}$ ramp (in the case of power on), the type of crystal and its electrical characteristics (if a quartz crystal is used), as well as any other external circuitry (e.g. capacitors), and the characteristics of the oscillator itself under the existing ambient conditions.

Once a clock is detected, the Wake-up Timer counts a fixed number of clocks (4,096), then sets the flag (OSCSTAT bit in the SCS register) that indicates that the main oscillator is ready for use. Software can then switch to the main oscillator and start any required PLLs. Refer to the Main Oscillator description in this chapter for details.

4.10 External clock output pin

For system test and development purposes, any one of several internal clocks may be brought out on the CLKOUT function available on the P1.27 pin, as shown in [Figure 12](#).

Clocks that may be observed via CLKOUT are the CPU clock (cclk), the main oscillator (osc_clk), the internal RC oscillator (irc_osc), the USB clock (usb_clk), and the RTC clock (rtc_clk).

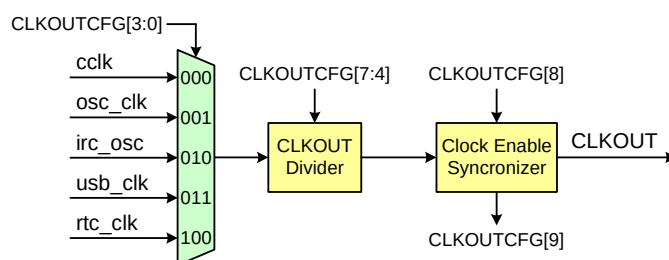


Fig 12. CLKOUT selection

4.10.1 Clock Output Configuration register (CLKOUTCFG - 0x400F C1C8)

The CLKOUTCFG register controls the selection of the internal clock that appears on the CLKOUT pin and allows dividing the clock by an integer value up to 16. The divider can be used to produce a system clock that is related to one of the on-chip clocks. For most clock sources, the division may be by 1. When the CPU clock is selected and is higher than approximately 50 MHz, the output must be divided in order to bring the frequency within the ability of the pin to switch with reasonable logic levels.

Note: The CLKOUT multiplexer is designed to switch cleanly, without glitches, between the possible clock sources. The divider is also designed to allow changing the divide value without glitches.

Table 47. Clock Output Configuration register (CLKOUTCFG - 0x400F C1C8) bit description

Bit	Symbol	Value	Description	Reset value
3:0	CLKOUTSEL		Selects the clock source for the CLKOUT function.	0
		0000	Selects the CPU clock as the CLKOUT source.	
		0001	Selects the main oscillator as the CLKOUT source.	
		0010	Selects the Internal RC oscillator as the CLKOUT source.	
		0011	Selects the USB clock as the CLKOUT source.	
		0100	Selects the RTC oscillator as the CLKOUT source.	
		others	Reserved, do not use these settings.	

Table 47. Clock Output Configuration register (CLKOUTCFG - 0x400F C1C8) bit description

Bit	Symbol	Value	Description	Reset value
7:4	CLKOUTDIV		Integer value to divide the output clock by, minus one.	0
		0000	Clock is divided by 1.	
		0001	Clock is divided by 2.	
		0010	Clock is divided by 3.	
		...	...	
		1111	Clock is divided by 16.	
8	CLKOUT_EN		CLKOUT enable control, allows switching the CLKOUT source without glitches. Clear to stop CLKOUT on the next falling edge. Set to enable CLKOUT.	0
9	CLKOUT_ACT		CLKOUT activity indication. Reads as 1 when CLKOUT is enabled. Read as 0 when CLKOUT has been disabled via the CLKOUT_EN bit and the clock has completed being stopped.	0
31:10	-		Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA